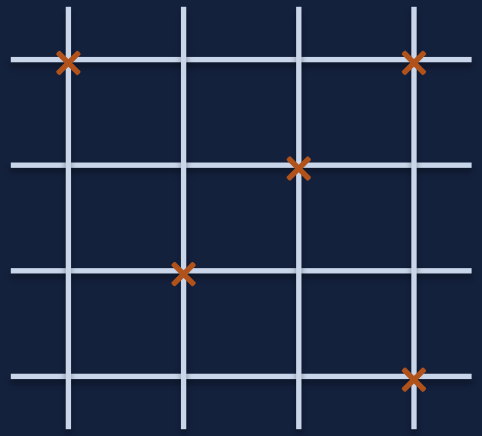
ENES 246 · DIGITAL LOGIC DESIGN

Lecture 27

Programmable Logic



ROM as decoder plus OR array · designing with a ROM · PLA and its programming table · PAL · sequential programmable devices and FPGAs

Reading: Mano & Ciletti, *Digital Design with an Introduction to the Verilog HDL*, 6th ed. — Chapter 7, §7.5–7.8

© 2026 Dr. Stefan Cular. All rights reserved.

Today’s roadmap

1 §7.5

ROM

Fixed AND, programmable OR

2 §7.6

PLA

Both arrays programmable

3 §7.7

PAL

Programmable AND, fixed OR

4 §7.8

Sequential PLDs

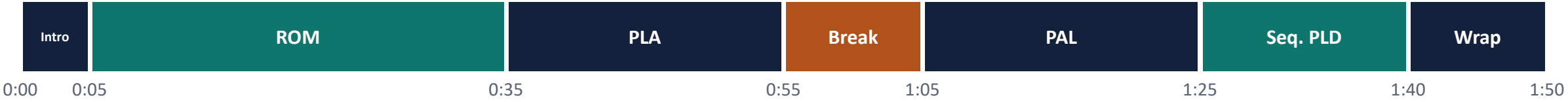
Macrocells to FPGAs

By the end of class you can…

- Describe a ROM as a decoder and an OR array
- Implement a combinational circuit with a ROM
- Write a PLA programming table with shared product terms
- Fit functions into a PAL with a fixed number of terms
- Explain the macrocell and the FPGA logic block

© 2026 Dr. Stefan Cular. All rights reserved.

Agenda for today's session



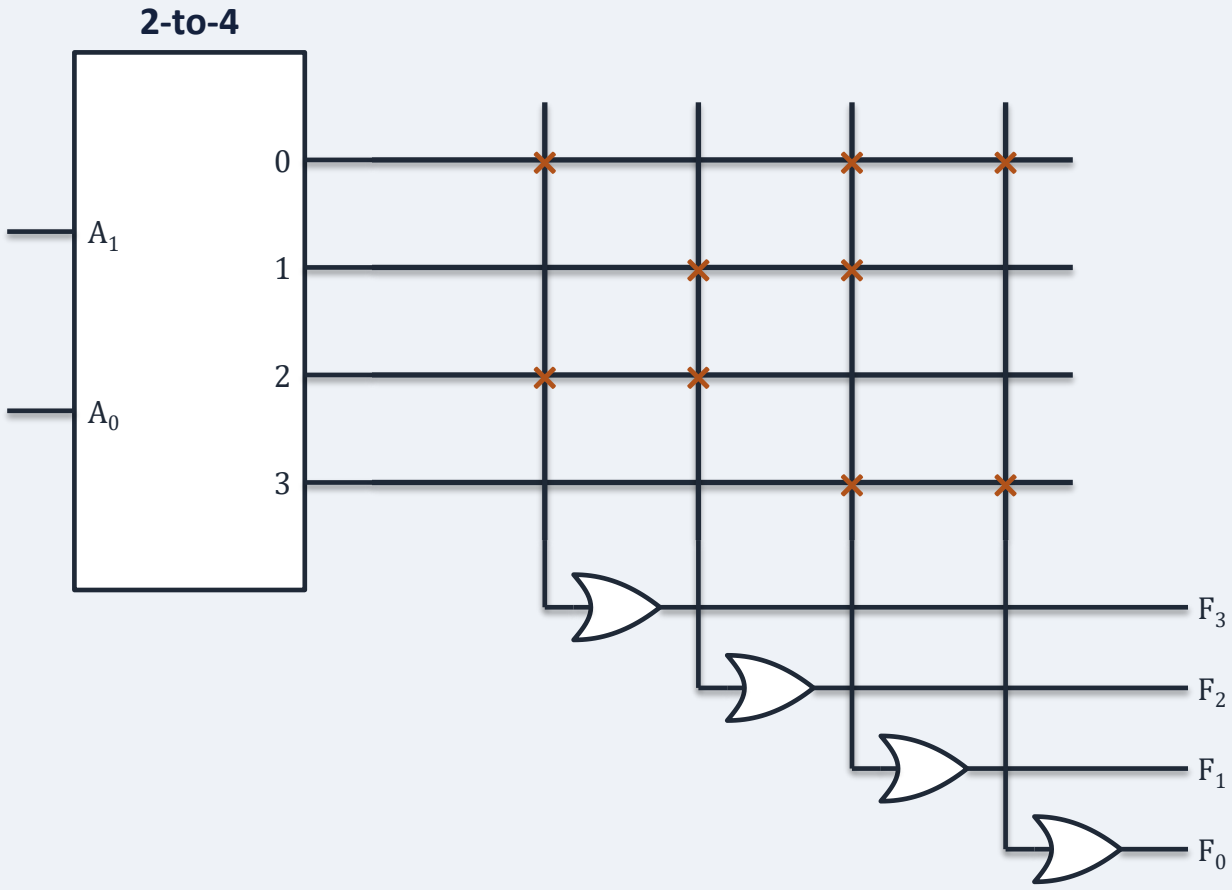
Elapsed time	Topic	In-class work
0:00 – 0:05	Welcome, roadmap	—
0:05 – 0:35	ROM structure and design	Examples 1, 2
0:35 – 0:55	The programmable logic array	Example 3
0:55 – 1:05	Break	
1:05 – 1:25	Programmable array logic	Example 4 · Your turn
1:25 – 1:40	Macrocells, CPLDs and FPGAs	—
1:40 – 1:50	Comparison; concept check; wrap-up	—

© 2026 Dr. Stefan Cular. All rights reserved.

Read-only memory

§7.5

A 4 × 4 ROM: decoder + programmable OR array



A_1A_0	$F_3F_2F_1F_0$
00	1011
01	0110
10	1100
11	0011

Reading the array

Each × is a 1 in the truth table. The decoder is fixed; only the OR connections are programmed.

© 2026 Dr. Stefan Cular. All rights reserved.

Example 1 — A squarer in a ROM

§7.5

EXAMPLE

Store the square of a 3-bit number $A_2A_1A_0$ (0–7) in a ROM. What is the smallest ROM?

A	A^2	$B_5B_4B_3B_2B_1B_0$	ROM word
000	0	000000	0000
001	1	000001	0000
010	4	000100	0001
011	9	001001	0010
100	16	010000	0100
101	25	011001	0110
110	36	100100	1001
111	49	110001	1100

1. 3-bit input: 8 words; $7^2 = 49$ needs 6 output bits
2. Fill the table: square each A, write it in 6 bits
3. Column B_0 equals A_0 (odd squares are odd): a wire
4. Column B_1 is 0 on every row: tie it to ground
5. Only $B_5 \ldots B_2$ need storing: an **8 × 4** ROM
6. 32 bits instead of 8 × 6 = 48

© 2026 Dr. Stefan Cular. All rights reserved.

ROM types

§7.5

Type	How it is programmed	Change it?
Mask ROM	by the manufacturer’s last mask	never
PROM	user blows fuses once	never
EPROM	charge on floating gates	erase with ultraviolet light
EEPROM	floating gates, electrical erase	byte by byte, in circuit
Flash	EEPROM erased in blocks	block by block: USB drives, SSDs, FPGA configuration

All are nonvolatile: data survives power loss, unlike the RAM of Lecture 25.

© 2026 Dr. Stefan Cular. All rights reserved.

Example 2 — Size a ROM

§7.5

EXAMPLE (a) A ROM implements a BCD-to-seven-segment decoder. Size? (b) A 1K × 8 ROM: address and data lines?

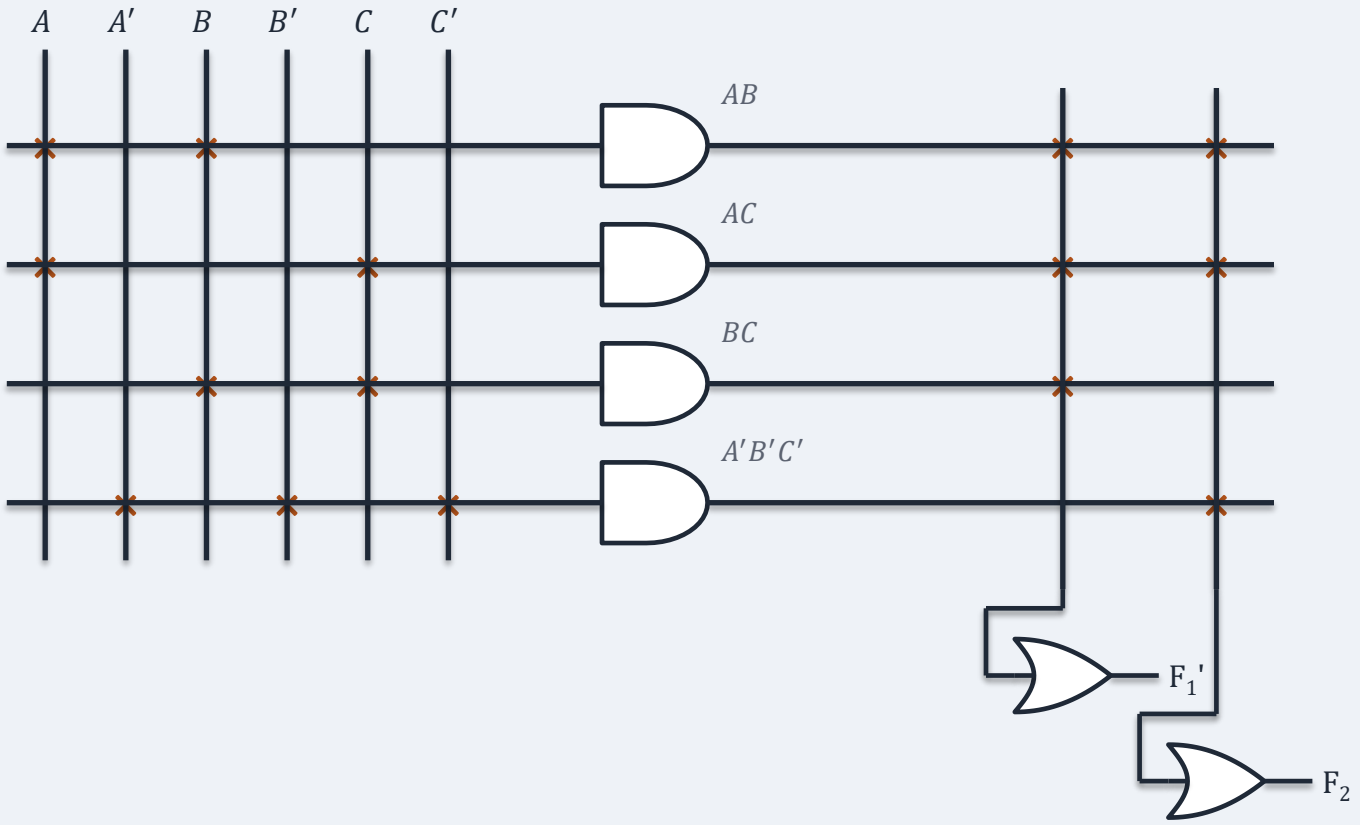
1. (a) 4 BCD inputs → $2^4 = 16$ words (address lines = inputs)
2. 7 segments → 7 bits per word
3. A **16 × 7** ROM; words 10–15 can hold a blank or error pattern
4. (b) $1K = 1024 = 2^{10}$ → **10** address lines
5. 8-bit words → **8** data lines
6. Capacity $2^{10} \times 8 =$ **8192** bits
7. A ROM does no minimization: every input row is stored

© 2026 Dr. Stefan Cular. All rights reserved.

The programmable logic array

§7.6

Programmable AND and OR arrays



PLA

Only the product terms needed are built, and outputs share them.

Each output may be inverted (XOR with a programmable bit), so either F or F' can be used.

Size: inputs × products × outputs.

© 2026 Dr. Stefan Cular. All rights reserved.

Example 3 — Programming a PLA

§7.6

EXAMPLE Implement $F_1 = \Sigma m(0,1,2,4)$ and $F_2 = \Sigma m(0,5,6,7)$ with the fewest product terms.

Product	A B C	F_1 (C)	F_2 (T)
AB	1 1 -	1	1
AC	1 - 1	1	1
BC	- 1 1	1	–
$A'B'C'$	0 0 0	–	1

C = complement the output, T = true.

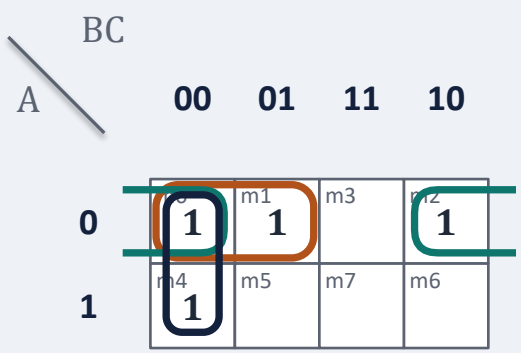
'1' in the product columns means the term feeds that OR gate.

1. Map F_1 and F_1': $F_1 = A'B' + A'C' + B'C'$, $F_1' = AB + AC + BC$
2. Map F_2: $F_2 = AB + AC + A'B'C'$
3. Choose forms that share terms: use F_1'
4. F_1' uses AB, AC, BC; F_2 uses $AB, AC, A'B'C'$
5. AB and AC are shared: **4 products** in all
6. Program output 1 as C (complement) → F_1; output 2 as T
7. Checked on all 8 rows ✓

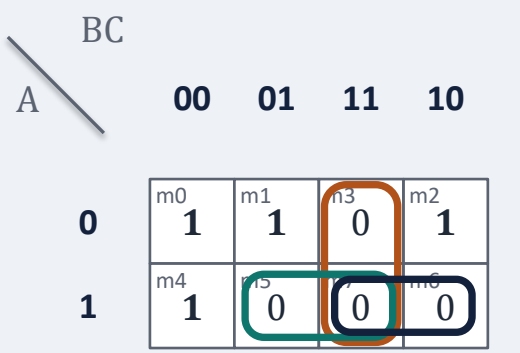
© 2026 Dr. Stefan Cular. All rights reserved.

Example 3 — The four K-maps

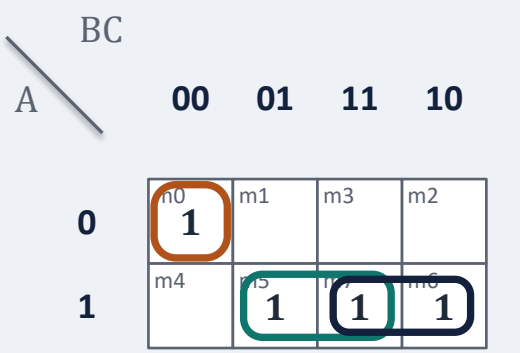
§7.6



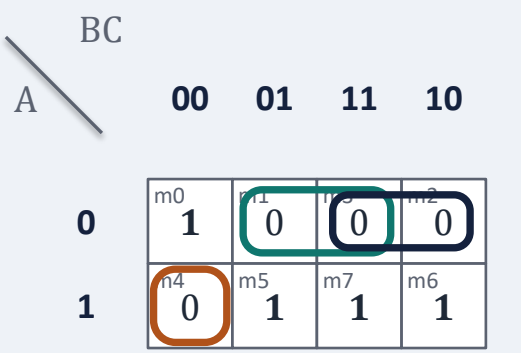
$F_1 = A'B' + A'C' + B'C'$



$F_1' = AB + AC + BC$



$F_2 = AB + AC + A'B'C'$



$F_2' = A'B + A'C + AB'C'$

1. Group the 1s: three pairs
2. None of them appears in F_2
3. Group the 0s: three pairs
4. AB and AC also appear in F_2
5. Group the 1s: two pairs and m0 alone
6. Use this form: it shares AB, AC
7. Group the 0s: two pairs and m4 alone
8. Shares nothing with F_1'

Best pair: F_1' and F_2 share AB and AC → 4 products. F_1 and F_2 need 6.

© 2026 Dr. Stefan Cular. All rights reserved.

Break

10 minutes — we resume at the 1:05 mark

Next: PAL, sequential PLDs and FPGAs

© 2026 Dr. Stefan Cular. All rights reserved.

Programmable array logic

§7.7

PAL

Programmable AND array, fixed OR array: each output owns a fixed number of product terms.

Terms cannot be shared between outputs.

An output can be fed back as an input to build larger functions.

Cheaper and faster than a PLA; the basis of the 22V10 and of CPLD macrocells.

	AND array	OR array
ROM	fixed (decoder)	programmable
PLA	programmable	programmable
PAL	programmable	fixed

Fitting a function

Minimize each output separately. If it needs more terms than its OR gate has, split it and feed part back.

© 2026 Dr. Stefan Cular. All rights reserved.

Example 4 — Fitting a PAL

§7.7

EXAMPLE

A PAL has 4 inputs and 3 product terms per output. Fit $z = ABC' + A'B'CD' + AC'D' + A'B'C'D$ (4 terms).

Output	Products	Uses
w	$ABC' + A'B'CD'$	2 of 3 terms
z	$w + AC'D' + A'B'C'D$	3 of 3 terms

1. z has 4 product terms; each PAL OR takes only 3
2. Split: put $ABC' + A'B'CD'$ on output w (2 of 3)
3. Feed w back into the array as an input
4. $z = w + AC'D' + A'B'C'D$: 3 terms ✓
5. Cost: an extra pass through the array
6. Checked on all 16 rows

© 2026 Dr. Stefan Cular. All rights reserved.

Your turn — Choose a device

§7.6

YOUR TURN

≈ 10 min

(a) What size ROM implements a 4-bit × 4-bit multiplier?

(b) How many product terms does a PLA need for $F_1 = AB + AC$, $F_2 = AB + BC$, $F_3 = AC + BC + A'B'$?

(c) Could these fit a PAL with 2 terms per output?

Count distinct product terms for the PLA.

© 2026 Dr. Stefan Cular. All rights reserved.

Your turn — Solution

SOLUTION

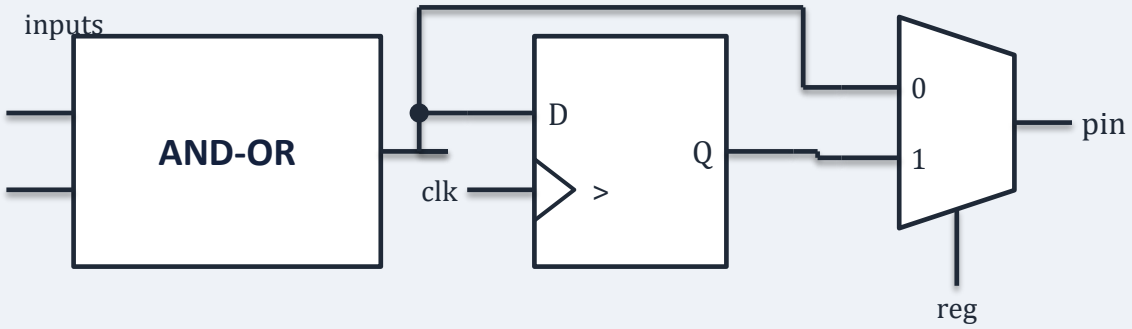
1. (a) A 4 × 4 multiplier has 8 inputs → $2^8 = 256$ words
2. The product needs $4 + 4 = 8$ bits → **256 × 8** (2048 bits)
3. (b) Products: $F_1: AB, AC;\ F_2: AB, BC$
4. $F_3: AC, BC, A'B'$
5. Distinct: $AB, AC, BC, A'B'$ → **4 products**
6. (c) F_1 and F_2 have 2 terms each: they fit
7. F_3 has 3 terms: one too many
8. Feedback: $w = AC + BC$ on a spare output
9. then $F_3 = w + A'B'$ ✓
10. Or use a PAL with 3 terms per output

© 2026 Dr. Stefan Cular. All rights reserved.

Sequential PLDs and FPGAs

§7.8

A macrocell



CPLD

Many PAL-like macrocells joined by a programmable switch matrix; configuration usually in on-chip flash.

FPGA

Thousands of logic blocks, each a small lookup table (a tiny ROM) plus a flip-flop.

Programmable routing, block RAM, multipliers, I/O blocks.

Configuration in SRAM, loaded from flash at power-up.

The Artix-7 on the Nexys board: about 100 000 logic cells.

© 2026 Dr. Stefan Cular. All rights reserved.

Concept check

1. Why can a ROM implement any function of its inputs?

2. What can a PLA do that a PAL cannot?

3. Why might a PLA implement F' instead of F?

4. What is inside an FPGA logic block?

© 2026 Dr. Stefan Cular. All rights reserved.

Key takeaways

ROM

fixed AND · prog. OR

$2^k \times n$ ROM

n functions of k vars

PLA

prog. AND · prog. OR

PAL

prog. AND · fixed OR

Macrocell

AND-OR + flip-flop

FPGA

LUTs + FFs + routing

Programmable logic is arrays of AND and OR you wire by choice.

Homework #11 due at the start of Lecture 28

Next: final exam preparation: memory and programmable logic review

© 2026 Dr. Stefan Cular. All rights reserved.